SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: CSA04 COURSE CODE: Operating Systems

COURSE OUTCOME COVERED

CO1: *Apply the concepts of Operating System types, structures, and system calls to demonstrate their functions in various computing environments. (BL2, BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5 **Total Marks:25**

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I M.Gowtham /192373060 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M.Gowtham

Annexure A

Short Answer Test

Question 1 **10 Marks**

A university computer laboratory allows multiple students to execute programs simultaneously on a shared Linux server.

(a) Identify the type of Operating System suitable for this environment and justify your answer.

(b) State any two responsibilities of the kernel in managing multiple user processes.

(c) Write a C-based system call sequence (or pseudo-code) to create a child process and display the Process IDs of both the parent and child.

Question 2 **10 Marks**

An application needs to store and retrieve student records from a text file.

(a) List the system calls required for file creation, writing, reading, and closing.

(b) Explain the purpose of any two file management system calls.

(c) Write a C-based system call sequence (or pseudo-code) to create a file, write a student name, and close the file.

Question 3 **10 Marks**

A user wants to copy the contents of one file to another.

(a) List the system calls required to complete this operation.

(b) Explain the purpose of any two of these system calls.

(c) Write a pseudo-code or C-based system call sequence to copy the contents of one file to another.

Question 4 **10 Marks**

A program needs to execute another program after creating a child process.

(a) State the purpose of the fork() and exec() system calls.

(b) Explain the role of the wait() system call.

(c) Write a system call sequence using fork(), exec(), and wait().

Question 5 **10 Marks**

An operating system manages multiple files stored on secondary memory.

(a) Explain the importance of file management in an operating system.

(b) State the purpose of the open() and close() system calls.

(c) Write a C-based system call sequence to open a file, read its contents, and close it.

RUBRICS FOR CALCULATION

Criteria	Weightage	*Level 4* (Exemplary)	*Level 3* (Proficient)	*Level 2* (Developing)	*Level 1* (Beginning)
Conceptual Understanding	35%	Demonstrates clear, complete, and in-depth understanding of OS concepts (types, structure, system calls)	Good understanding with minor gaps	Understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answers	25%	All answers are correct, precise, and relevant	Mostly correct with minor errors	Partially correct with noticeable errors	Mostly incorrect or irrelevant
Application of Knowledge	15%	Effectively applies concepts to scenarios/questions	Applies with minor mistakes	Limited or partial application	No meaningful application
Completeness of Response	15%	All parts of each question are fully answered	Most parts covered with small omissions	Some parts missing	Incomplete answers
Clarity & Presentation	10%	Clear, well-structured, and neatly presented answers	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Marks Evaluation:

Question No.	Conceptual Understanding (35%)	Accuracy of Answers (25%)	Application of Knowledge (15%)	Completeness of Response (15%)	Clarity & Presentation (10%)	Total Marks (100%)
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

1) Multiple students using a shared Linux server

(a) Identify the type of operating system and justify your answer

A multi-user operating system is suitable for this environment. It allows multiple students to access and execute programs on the same Linux server simultaneously while managing system resources efficiently.

(b) State any two responsibilities of the kernel.

1) Process Management: The kernel creates, schedules, and terminates processes and allocates CPU time to them.

2) Memory Management: The kernel allocates and deallocates memory for processes and prevents unauthorized access to another process's memory.

(c) C program to create a child process and display Process IDs.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main(){
    pid_t pid = fork();
    if (pid == 0){
        Printf ("Child process ID: %d\n", getpid());
        Printf ("Parent process ID: %d\n", getppid());
    }else if (pid > 0){
        Printf("Parent Process ID: %d\n", getpid());
        Printf ("Child Process ID: %d\n", pid);
    }else{
```

```
    perror("fork failed");
}
return 0;
}
```

Result: The program creates a child process using fork() and displays the process IDs of both parent and child.

2) File creation, writing, Reading and closing

(a) List the required system calls.

* open() - opens or creates a files
* write() - writes data into a file
* read() - Reads data from a file
* close() - closes an opened file.

(b) Explain the purpose of any two system calls.

1) open(): opens an existing files or creates a new file when used with the appropriate flags, such as O_CREAT.

2) write(): writes the specified number of bytes from a buffer into a file.

(c) C Program to create a file, write a student name, and close it.

```
#include<stdio.h>
#include<fcntl.h>
#include<unistd.h>
int main(){
    int fd=open("student.txt", O_WRONLY|O_CREAT|O_TRUNC,
                                                0644);
    if(fd==-1){
```

```
    Perror("open failed");
    return 1;
}
char name[] = "Rahul";
if (write(fd, name, sizeof(name)-1) == -1){
    Perror("write failed");
    close(fd);
    return 1;
}
    close(fd);
    Printf("student name written successfully.\n");
    return 0;
}
```

Result: The file student.txt is created and the student name is written into it.

3) Copy the contents of one file to Another

(a) List the required system calls.

1) open() - opens the source file and destination file.
2) Read() - Reads data from the source file.
3) Write() - Writes data to the destination file.
4) close() - closes both files.

(b) Explain the purpose of any two system calls.

1. read(): Reads bytes from a file into a memory buffer.
2. Write(): Transfers bytes from a memory buffer into a file.

(c) C program to copy one file to another

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
int main(){
    char buffer[1024];
    ssize_t n;
    int src = open("source.txt", O_RDONLY);
    if (src == -1){
        Perror("source file error");
        return 1;
    }
    int dest = open("destination.txt", O_WRONGLY | O_CREAT | O_TRUNC,
                                                    0644);
    if (dest == -1){
        Perror("Destination file error");
        close(src);
        return 1;
    }
    while ((n = read(src, buffer, sizeof(buffer))) > 0){
        ssize_t written = 0;
        while (written < n){
            ssize_t w = write(dest, buffer + written, n - written);
            if (w < 0){
                Perror("write failed");
                close(src);
                close(dest);
                return 1;
            }
            written += w;
        }
    }
    if (n == -1)
```

```
if (n==-1)
    Perror ("read failed");
else
    Printf ("File copied successfully. \n");
close (src);
close (dest);
return n==-1 ?1:0;
}
```

Result: The contents of source.txt are copied to destination.txt. Both files are closed after the operation.

4) Executing Another Program using fork(), exec(), and wait()

(a) state the purpose of fork() and exec().

* fork(): Creates a new child process by duplicating the calling process.
* exec(): Replaces the current process's program with another program. The process ID generally remains unchanged.

(b) Explain the role of wait().

The wait() system call makes the parent process wait until a child process terminates. It also collects the child's termination status and helps prevent zombie processes.

(c) C program using fork(), exec(), and wait()

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>
int main(){
```

```
pid_t pid = fork();
if (pid == 0) {
    Printf("child process executing another program. \n");
    execlp("ls", "ls", "-l", (char *)NULL);
    Perror("exec failed");
    exit(1);
} else if (pid > 0) {
    wait(NULL);
    Printf("child process completed. \n");
} else {
    Perror("fork failed");
    return 1;
}
return 0;
}
```

Result: The child process executes the ls -l command, and the parent waits until the child finishes.

5) File Management in an Operating system

(a) Explain the importance of file management.

File management is an important function of an operating system that organizes, stores, retrieves, and protects files on secondary storage devices. It helps users access data efficiently, manage directories, control file permissions, and use storage space effectively.

(b) state the purpose of open() and close()

* open(): opens a file and returns a file descriptor used for file operations.

* close(): Releases the file descriptor and associated

resources when file operations are completed.

(c) C Program to open a file, read its contents, and close it.

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
int main(){
   char buffer[1024];
   ssize_t n;
   int fd = open("student.txt", O_RDONLY);
   if(fd==-1){
      Perror("File open failed");
      return 1;
   }
   while((n = read(fd, buffer, sizeof(buffer)))>0){
      if(write(STDOUT_FILENO, buffer, n)==-1){
         Perror("output failed");
         close(fd);
         return 1;
      }
   }
   if(n==-1){
      Perror("Read failed");
      close(fd);
      return n==-1?1:0;
}
```

Result: The program opens student.txt, reads and displays its contents, and closes the file.